Compte Rendu final de Projet de Recherche

Pierre Baumann, Ivan Mulot-Radojcic, Damien Assire

10 mai 2024

Résumé

Abstract

Table des matières

1 Introduction **1**

2 Génération du graphe **2**
2.1 Transformation du graphe 2

3 Génération des attaques **4**
3.1 Attaques statiques 4
3.1.1 Attaque aléatoire statique 4
3.1.2 Attaque par centralité intermédiaire 5
3.2 Attaques dynamiques 5
3.2.1 Attaque complètement aléatoire 5
3.2.2 Attaque aléatoire mouvante 6
3.2.3 Attaque par centralité intermédiaire mouvante 6

sections

1 Introduction

Dans notre projet, nous cherchons à modéliser et évaluer l'impact d'une manifestation sur une ville. Pour modéliser ce problème nous utiliserons une variante des graphes, les graphes dynamiques.

Un graphe est un objet informatique constitué de sommets reliés par des arrêtes. Les graphes permettent de modéliser de nombreux problèmes, notamment en informatique, en biologigie, en sociologie, physique,...

Un graphe peut être défini par $G = (V, E)$ où V est l'ensemble des sommets et $E \subseteq V^2$ est l'ensemble des liens $e = (u, v, w)$ $u, v \in V, w \in \mathbb{N}$ qui est la pondération du lien de u à v, noté u $\xrightarrow{w}$ v. Les graphes seront également orienté, i.e si $(u, v, w) \in E$ alors il est possible d'accéder à v depuis u mais pas forcément d'accéder à u depuis v, à moins que $\exists\ (v, u, w') \in E$.

Un graphe dynamique est alors un graphe dont les sommets et les arrêtes peuvent évoluer dans le temps. Le modèle de graphe dynamique que nous utilisons peut être décrit comme un triplet $G = (V, E, T)$ où $T = \{(t, E_t) \mid t \in [\![0, t_{max}]\!] \text{ et } E_t \subseteq E\}$ est l'ensemble des instants auxquelles des liens entre les sommets sont supprimés.

référence pour graphe dynamique

Nous utilisons également diamètre d'un graphe, définie par :

$$diametre = \max_{\forall u,v \in V} \{\text{longueur}(PlusCourtChemin(u, v))\}$$ [1]

1. Cette formule ne marche que si le graphe est connexe, sinon il faut prendre le maximum des diamètres des composantes connexes.

La mesure classique pour déterminer à quel point il est facile de se déplacer dans un réseau est l'efficacité. L'efficacité d'un graphe est définie comme la moyenne des inverses des distances entre chaque paire de sommets.

$$E(G) = \frac{1}{N(N-1)} \sum_{i \neq j} \frac{1}{d(v_i, v_j)} \tag{1}$$

avec $d(v_i, v_j)$ la distance entre deux sommets v_i et v_j, N le nombre de sommets du graphe.

référence pour efficacité

Notre travail est alors de déterminer une mesure de l'efficacité dans un graphe dynamique et de trouver des stratégies pour attaquer ce graphe et réduire cette efficacité en prenant en compte que le graphe est dynamique et va évoluer dans le temps.

2 Génération du graphe

Nous représentons les villes sous la forme d'un graphe. Les données sont importées depuis une bibliothèque de données de rues nommées OpenStreetMap [2]. Pour cela, nous utilisons la librairie Python OSMnx [3] qui les convertis sous la forme d'un graphe.

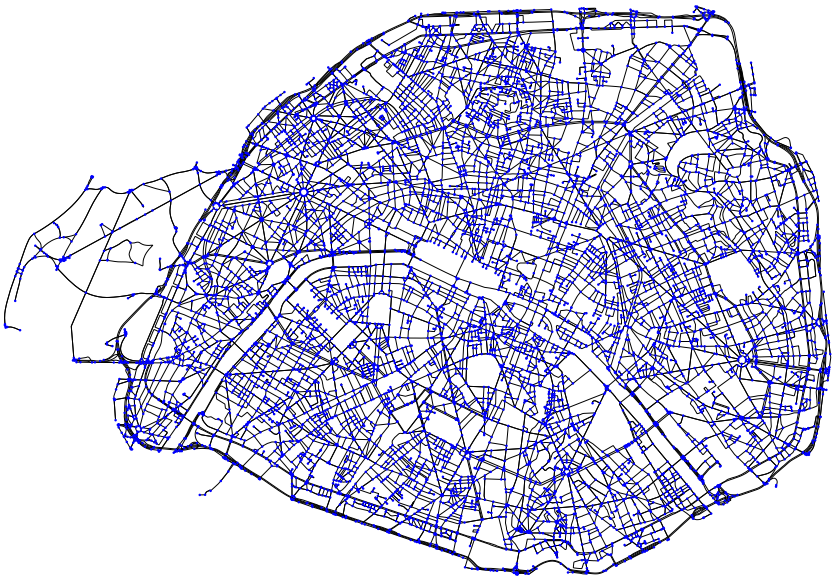


FIGURE 1 – Graphe de la ville de Paris

2.1 Transformation du graphe

Nous avons choisi de représenter le graphe sous la forme d'une matrice creuse en format CSR (Compressed Sparse Row). Sous cette représentation, le graphe est représenté par trois tableaux :

— ROW_INDEX qui contient les bornes des intervalles de valeurs liés aux liens partant d'un nud u.

$$[\![ROW_INDEX[u], ROW_INDEX[u+1] - 1]\!]$$

Si u n'a aucun lien sortant, $ROW_INDEX[u] = ROW_INDEX[u+1]$.

— COL_INDEX contient le sommet destination d'un lien à l'index donné

— V contient le poids du lien.

2. https://www.openstreetmap.org
3. https://osmnx.readthedocs.io/en/stable/

Lors de la récupération des données, OSMnx renvoie un multi-graphe orienté.

Nous transformons le graphe car :

1. Nous utilisons des distances temporels et non la distance physique entre les deux noeuds.
2. La ville est représentée sous la forme d'un multigraphe alors que les simulations sont faites sur un graphe avec au plus un lien d'un sommet u à v.

Pour cela, nous utilisons les fonctions des librairies NetworkX et SciPy afin d'obtenir les tableaux ROW_INDEX et COL_INDEX. Le tableau V doit est calculé via l'algorithme 1

Algorithm 1 Calcul de V

Require : G : multi-graphe, ROW_INDEX, COL_INDEX
  for $i \in [\![0, G.NumNodes]\!]$ **do**
    for $j \in ROW_INDEX[i, i+1]$ **do**
      $weigth \leftarrow \emptyset$
      for all $e \in G.edges[i][COL_INDEX[j]]$ **do**
        $weigth.append\left(\frac{e.length}{e.maxspeed}\right)$
      end for
      $V[j] \leftarrow min(weigth)$
    end for
  end for

L'algorithme 1 parcourt l'ensemble des couples $(u, v) \in V^2$ tels qu'il existe au moins un lien de u à v dans E. Après, il parcourt tout les liens de u à v et calcul le minimum des distances temporelles, donnés par la formule

$$\Delta t = \frac{d}{v}$$

Où la durée Δt est la distance temporelle, d la longueur du lien et v la vitesse maximale. Dans le cas où il y a plusieurs liens entre deux sommets u et v, on garde le lien de poids le plus petit car c'est celui-ci qui sera emprunter lors d'un plus court chemin. Certaines données nécessaires aux calculs peuvent êtres manquantes et sont remplacées par des valeurs par défaut pour la vitesse max, et la distance euclidienne de u à v pour la longueur du lien.

La figure 2 montre le graphe résultant de cette transformation sur un sous-ensemble du graphe de Paris correspondant au cinquième arrondissement.

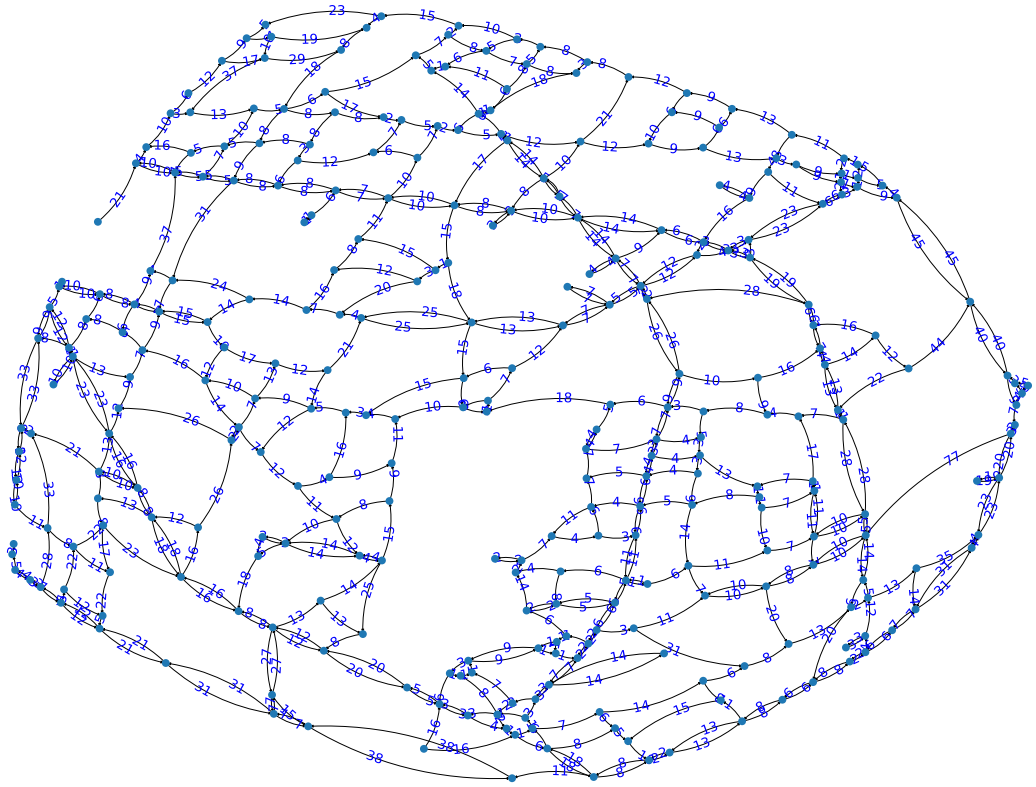


FIGURE 2 – Transformation en graphe temporel du 5e arrondissement de Paris

3 Génération des attaques

À chaque pas de temps t_i, nous allons attaquer le graphe en bloquant des liens jusqu'à un certain budget. Nous avons choisi de mettre le prix de chaque lien à 1, car il est plus simple d'interpréter les résultats avec un prix constant.

Nous avons choisi de séparer la gestion des attaques des stratégies d'attaques. Pour la gestion, nous avons la classe Attaque, avec 2 primitives :

— Bloquer un lien à un temps donné

— Retranscrire l'attaque dans un fichier pour la simulation

Comme ça, les stratégies peuvent se focaliser sur l'algorithme de choix des liens à supprimer, et la classe Attaque se charge de faire le reste.

Nous classons les stratégies d'attaques en deux catégories : les attaques dynamiques et les attaques statiques.

3.1 Attaques statiques

Au contraire des attaques dynamiques, les attaques statiques sont des attaques où un lien bloqué est bloqué de t_0 à $t_{\max}$.

3.1.1 Attaque aléatoire statique

Cette première stratégie consiste à bloquer des liens aléatoirement, et rester sur ces liens pour toute la durée de l'attaque.

Algorithm 2 Attaque aléatoire statique

```
Liens ← Graphe.liens()
LiensBloques ← Mélanger(Liens)
for lien ← LiensBloques do
    Attaque.bloquer(lien, t)
end for
```

3.1.2 Attaque par centralité intermédiaire

Dans l'état-de-l'art, l'attaque par centralité intermédiaire est considérée comme la stratégie la plus efficace. La centralité intermédiaire est une mesure représentant l'importance d'un lien dans un graphe. Elle est calculée en comptant le nombre de plus courts chemins passant par ce lien.

Cette stratégie consiste donc à bloquer les liens les plus importants en les triant par centralité intermédiaire. Nous notons d'ailleurs qu'OSMnx contient des options pour récupérer les centralités intermédiaires des liens d'un graphe directement.

Algorithm 3 Attaque par centralité intermédiaire

```
Liens ← Graphe.liens()
LiensTries ← TrierParCentralité(Liens)
for t ← 0 to TempsMax do
    BudgetRestant ← Budget
    for lien ← LiensTries do
        Attaque.bloquer(lien, t)
        Prix ← lien.prix()
        BudgetRestant ← BudgetRestant − Prix
        if BudgetRestant ≤ 0 then
            break
        end if
    end for
end for
```

Remarque : cette stratégie est en fait une stratégie générique, qui peut être utilisée pour trier les liens par n'importe quel critère, comme le nombre de voisins, le nombre de voies composant la rue, etc... Nous l'utilisons avec la centralité intermédiaire, car c'est le critère le plus couramment utilisé dans la littérature.

3.2 Attaques dynamiques

Les attaques dynamiques sont des attaques où un lien peut être bloqué puis débloqué plus tard, notamment, car les obstacles se déplacent au cours du temps. Nous avons choisi d'étudier des équivalents des attaques statiques, mais qui évoluent dans le temps, pour voir si elles sont plus efficaces.

3.2.1 Attaque complètement aléatoire

Cette stratégie est la première adaptation dynamique de l'attaque aléatoire. Elle consiste à attaquer des liens aléatoirement, différents à chaque pas de temps. Cette attaque nous servira de base pour comparer les autres stratégies.

Algorithm 4 Attaque complètement aléatoire

```
for t ← 0 to TempsMax do
    LiensRestants ← Mélanger(Graphe.liens())
    BudgetRestant ← Budget
    while BudgetRestant > 0 ∧ LiensRestants ≠ ∅ do
        lienABloquer ← Liens.pop()
        Attaque.bloquer(lienABloquer, t)
        Prix ← lienABloquer.prix()
        BudgetRestant ← BudgetRestant − Prix
    end while
end for
```

3.2.2 Attaque aléatoire mouvante

Pour cette stratégie, nous commençons par une attaque aléatoire pour le premier pas de temps t_0, mais pour les suivants, les obstacles bloquant un lien vont se déplacer vers un lien voisin non bloqué s'ils le peuvent. Dans notre contexte de manifestations et de marches, cette stratégie est plus réaliste, car les manifestants vont se déplacer dans la ville.

Algorithm 5 Attaque mouvante

```
LiensOriginaux ← AttaqueAléatoire(Graphe, t_max=1, Budget)
for t ← 1 to t_max do
    NewLiens ← ∅
    for lien ← LiensOriginaux do
        Voisins ← lien.voisins()
        for v ← Voisins do
            if v ∉ NewLiens then
                NewLiens.bloquer(v)
                continue to next lien
            end if
        end for
        NewLiens.bloquer(lien)
    end for
    LiensOriginaux ← NewLiens
end for
```

3.2.3 Attaque par centralité intermédiaire mouvante

Cette stratégie consiste à bloquer les liens les plus importants en les triant par centralité intermédiaire, puis pour les prochains pas de temps, on calcule les nouvelles centralités intermédiaires sur le graphe avec les liens bloqués, et on bloque ces nouveaux liens les plus importants. Note : dû aux nombre de calculs nécessaires pour les centralités intermédiaires, nous les avons approximées en ne prennant que 20% des chemins les plus courts, ce qui est suffisant pour avoir une très bonne approximation.

Algorithm 6 Attaque par centralité intermédiaire mouvante

```
Liens ← Graphe.liens()
LiensTries ← TrierParCentralité(Liens)
LiensBloques ← ∅
for t ← 0 to TempsMax do
    BudgetRestant ← Budget
    for lien ← LiensTries do
        Attaque.bloquer(lien, t)
        LiensBloques.ajouter(lien)
        Prix ← lien.prix()
        BudgetRestant ← BudgetRestant − Prix
        if BudgetRestant ≤ 0 then
            break
        end if
    end for
    Graphe = Graphe.enlever(LiensBloques)
    LiensTries ← TrierParCentralité(Graphe.liens())
end for
```